

MODULE2–FLUTTERFONDAMENTAL

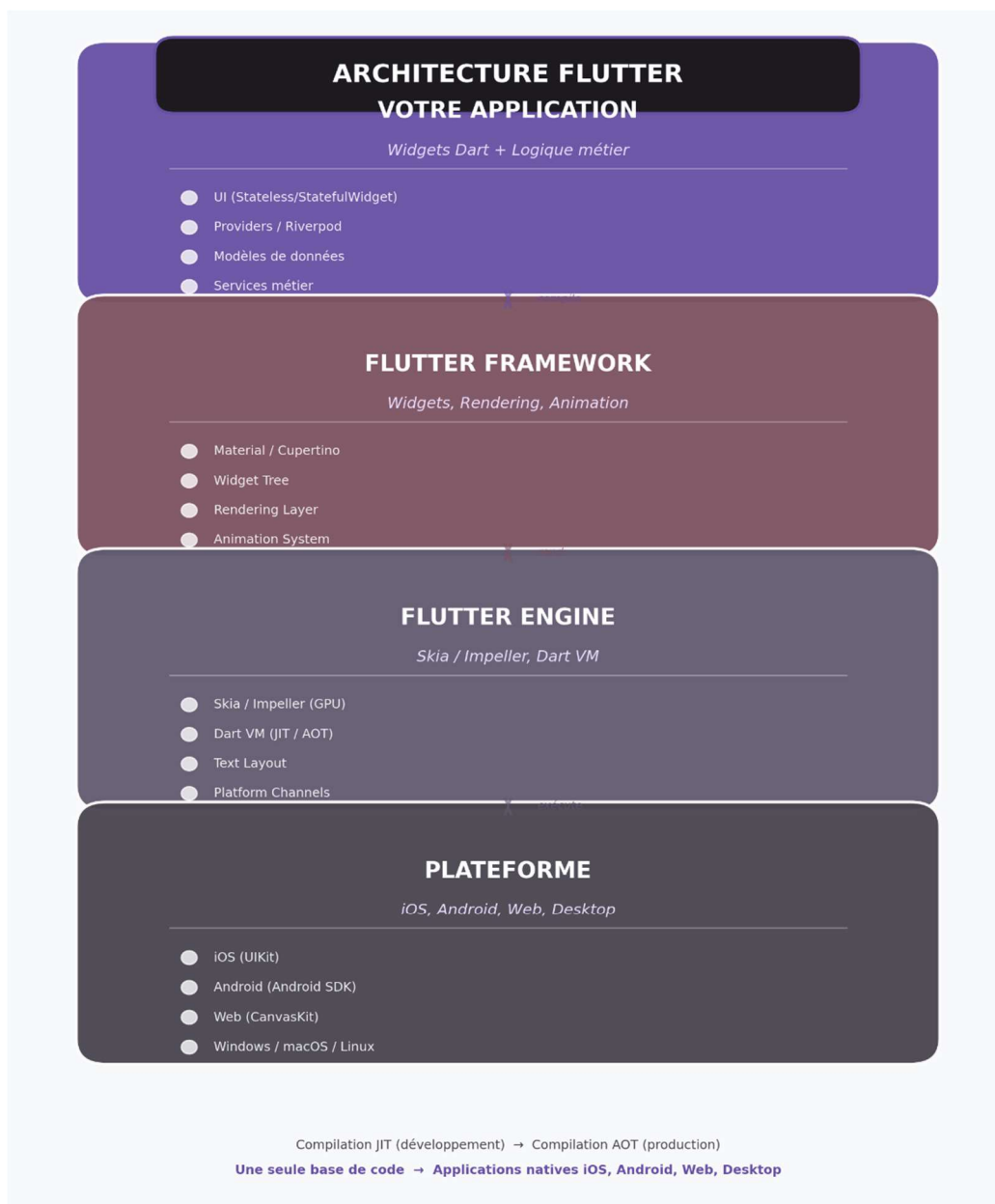
Formation Flutter Professionnelle

SUPPORT DE COURS – Théorie complète

1. Qu'est-ce que Flutter ?

Flutter est un **SDK (Software Development Kit)** open-source créé par Google pour développer des applications multiplateformes (iOS, Android, Web, Desktop) à partir d'une seule base de code.

Architecture unique de Flutter



Pourquoi Flutter est différent ?



- **Pas de WebView** : contrairement à React Native ou Ionic, Flutter ne repose pas sur le navigateur du système.
- **Pas de widgets natifs** : Flutter dessine chaque pixel à l'écran via son propre moteur de rendu (Skia/Impeller).
- **Performance native** : 60/120 FPS fluides grâce à la compilation AOT.

Compilation JIT vs AOT

Mode	Quand ?	Pourquoi ?
JIT (Just In Time)	Pendant le développement	Hot reload ultra-rapide (< 1 seconde)
AOT (Ahead Of Time)	Pour la production	Code machine natif, performance maximale

2. Installation de l'environnement

Prérequis système

- **Windows** : Windows 10 ou supérieur, 8 Go RAM minimum (16 Go recommandé), 2 Go d'espace disque.
- **macOS** : macOS 10.14 ou supérieur, Xcode pour le build iOS.
- **Linux** : Distribution 64-bit, librairies GTK.

Étapes d'installation

1. Télécharger Flutter SDK

Windows (via git)

```
git clone https://github.com/flutter/flutter.git -b stable
```

macOS/Linux

```
git clone https://github.com/flutter/flutter.git -b stable
```

2. Ajouter au PATH

Windows (PowerShell)

```
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\flutter\bin", "User")
```

macOS/Linux (ajouter dans ~/.zshrc ou ~/.bashrc)

```
export PATH="$PATH:/chemin/vers/flutter/bin"
```

3. Vérifier l'installation

```
flutter doctor
```

Sortie attendue :

```
[] Flutter (Channel stable, 3.x.x, ...)
```

```
[] Android toolchain - develop for Android devices
```

```
[] Chrome - develop for the web
```

```
[] Android Studio (version 2022.x)
```

```
[] VS Code (version 1.x)
```

[] Connected device (1 available)

Résolution des problèmes courants

Problème	Solution
Android SDK not found	Installer Android Studio → SDK Manager → SDK Platforms + SDK Tools
cmdline-tools component is missing	SDK Tools → Android SDK Command-line Tools (latest)
Unable to locate Android SDK	Définir ANDROID_HOME dans les variables d'environnement
No devices available	Démarrer un émulateur ou brancher un téléphone en mode développeur

4. IDE recommandés

- **Android Studio** : complet, émulateur intégré, outils de profiling.
- **VS Code** : léger, extensions Flutter/Dart excellentes.

Extensions VS Code indispensables :

- Flutter (Dart Code)
- Dart
- Awesome Flutter Snippets
- Flutter Tree

3. Structure d'un projet Flutter

```
mon_projet/
├── android/           # Configuration Android native
├── ios/               # Configuration iOS native
├── lib/               # CODE DART DE L'APPLICATION
│   ├── main.dart     # Point d'entrée
│   └── ...
├── test/              # Tests unitaires et widgets
├── pubspec.yaml       # Gestion des dépendances et assets
├── analysis_options.yaml # Règles d'analyse statique
└── README.md
```

Le fichier pubspec.yaml

```
name: mon_projet
description: Une application Flutter professionnelle
publish_to: 'none'
version: 1.0.0+1
```



```
environment:  
  sdk: '>=3.0.0 <4.0.0'
```

```
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.6  
  http: ^1.1.0  
  riverpod: ^2.4.0
```

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  flutter_lints: ^3.0.0
```

```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/  
    - assets/fonts/  
  fonts:  
    - family: Roboto  
      fonts:  
        - asset: assets/fonts/Roboto-Regular.ttf
```

Commandes essentielles

flutter create mon_projet	# Créer un projet
flutter run	# Lancer sur l'appareil connecté
flutter run --hot	# Hot reload actif (par défaut)
flutter build apk	# Build Android
flutter build ios	# Build iOS (macOS uniquement)
flutter pub get	# Télécharger les dépendances
flutter pub add nom_package	# Ajouter un package
flutter analyze	# Analyse statique du code
flutter test	# Lancer les tests

4. Le point d'entrée : main.dart

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(const MonApplication());  
}
```

```

class MonApplication extends StatelessWidget {
  const MonApplication({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Mon Application',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const PageAccueil(),
    );
  }
}

class PageAccueil extends StatelessWidget {
  const PageAccueil({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Accueil'),
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      ),
      body: const Center(
        child: Text('Hello Flutter !'),
      ),
    );
  }
}

```

Anatomie ligne par ligne :

- `import 'package:flutter/material.dart'` : importe la librairie Material Design.
- `void main()` : point d'entrée du programme.
- `runApp()` : attache l'application à l'écran.
- `MaterialApp` : widget racine qui configure le thème, la navigation, les locales.
- `Scaffold` : structure de base d'une page (`AppBar`, `body`, `drawer`, `FAB`).
- `AppBar` : barre d'application en haut.



- Center : centre son enfant dans l'espace disponible.
- Text : affiche du texte stylisé.

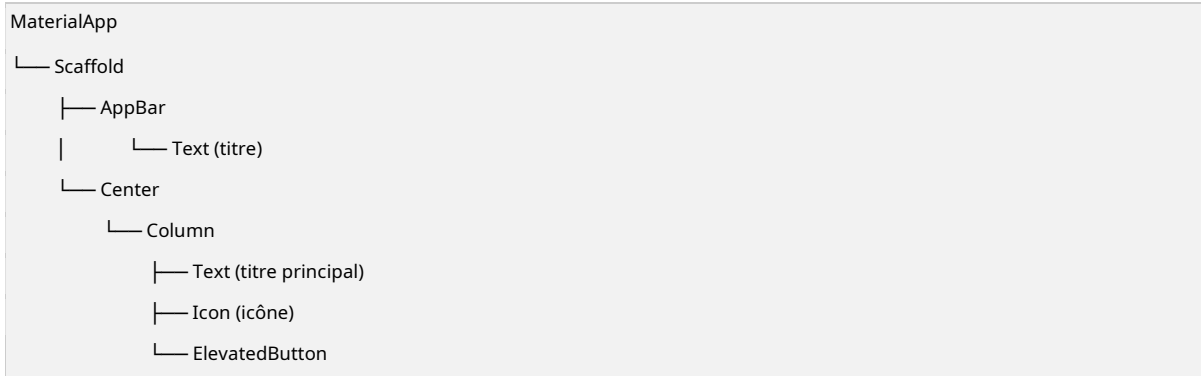
5. La philosophie : Everything is a Widget

En Flutter, **tout est un widget**. Un widget est une description immuable d'une partie de l'interface.

Types de widgets :

- **Widgets de mise en page** : organisent les autres widgets (Row, Column, Stack, Container, Padding).
- **Widgets de contenu** : affichent quelque chose (Text, Image, Icon).
- **Widgets d'interaction** : réagissent aux actions (ElevatedButton, GestureDetector, TextField).
- **Widgets de structure** : définissent la structure de la page (Scaffold, AppBar).

L'arbre des widgets (Widget Tree)



Chaque widget a un **parent** (sauf la racine) et peut avoir des **enfants**.

6. StatelessWidget vs StatefulWidget

StatelessWidget (Widget sans état)

- Description immuable.
- Ne change jamais après sa construction.
- Idéal pour : textes, icônes, images, layouts statiques.

```
class CarteBienvenue extends StatelessWidget {
  final String nom;

  const CarteBienvenue({super.key, required this.nom});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
```

```

padding: const EdgeInsets.all(16.0),
child: Text('Bienvenue, $nom !'),
),
);
}
}

```

StatefulWidget (Widget avec état)

- Peut changer d'apparence en réponse à des interactions ou des événements.
- Composé de deux classes : le widget (immuable) et l'état (mutable).
- Idéal pour : formulaires, animations, listes dynamiques, compteurs.

```

class Compteur extends StatefulWidget {
  const Compteur({super.key});

  @override
  State<Compteur> createState() => _CompteurState();
}

class _CompteurState extends State<Compteur> {
  int _compte = 0;

  void _incrémenter() {
    setState(() {
      _compte++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Valeur : $_compte'),
        ElevatedButton(
          onPressed: _incrémenter,
          child: const Text('Incrémenter'),
        ),
      ],
    );
  }
}

```

Cycle de vie d'un StatefulWidget

1. createState() → Création de l'état
2. initState() → Initialisation (une seule fois)



- 3. build() → Construction de l'UI
- 4. setState() → Demande de reconstruction
- 5. didUpdateWidget() → Le widget parent a changé
- 6. dispose() → Nettoyage avant destruction

Quand utiliser quoi ?

Situation	Widget
Texte statique, icône, image	StatelessWidget
Bouton simple (sans état interne)	StatelessWidget
Formulaire avec saisie	StatefulWidget
Situation	Widget
Liste qui change de contenu	StatefulWidget
Animation	StatefulWidget
Page avec données chargées depuis une API	StatefulWidget

Règle d'or : commencez toujours par un StatelessWidget. Passez à StatefulWidget uniquement si vous avez besoin de gérer un état qui change.

7. Widgets de base essentiels

Container Le couteau suisse du layout. Peut avoir une taille, une couleur, un padding, une marge, une décoration.

```
Container(  
  width: 200,  
  height: 100,  
  margin: const EdgeInsets.all(16),  
  padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(12),  
    boxShadow: [  
      BoxShadow(  
        color: Colors.black.withOpacity(0.2),  
        blurRadius: 8,  
        offset: const Offset(0, 4),  
      ),  
    ],  
  ),  
  child: const Text('Contenu'),  
)
```

Row et Column Organisent les widgets horizontalement (Row) ou verticalement (Column).

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center, // vertical  
  crossAxisAlignment: CrossAxisAlignment.stretch, // horizontal
```



```

children: [
  const Text('Ligne 1'),
  const SizedBox(height: 16), // espace fixe
  const Text('Ligne 2'),
],
)

```

Propriétés clés :

- `mainAxisAlignment` : alignement sur l'axe principal (vertical pour Column, horizontal pour Row)
- `crossAxisAlignment` : alignement sur l'axe transversal
- `mainAxisSize` : taille sur l'axe principal (max ou min)

Text

```

Text(
  'Titre',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.deepPurple,
    letterSpacing: 1.2,
  ),
)

```

Icon et Image

```

// Icône Material
const Icon(Icons.favorite, color: Colors.red, size: 48)

// Image depuis les assets
Image.asset('assets/images/logo.png')

// Image depuis le réseau
Image.network('https://example.com/image.jpg')

```

Important : pour utiliser une image locale, vous devez d'abord la déclarer dans `pubspec.yaml` :

```

flutter:
  assets:
    - assets/images/

```

8. Hot Reload et Hot Restart

Commande	Action	Quand l'utiliser ?
Hot Reload (r ou sauvegarde)	Met à jour l'UI en gardant l'état	Changements de layout, couleurs, textes



Hot Restart (R)	Redémarre l'app depuis le début	Changements dans main(), nouvelles variables globales
Full Restart (stop + run)	Recompile tout	Problèmes persistants, changements natifs

Astuce : si le hot reload ne reflète pas vos changements, vérifiez que vous n'avez pas modifié le main() ou des constantes globales. Dans ce cas, utilisez Hot Restart.

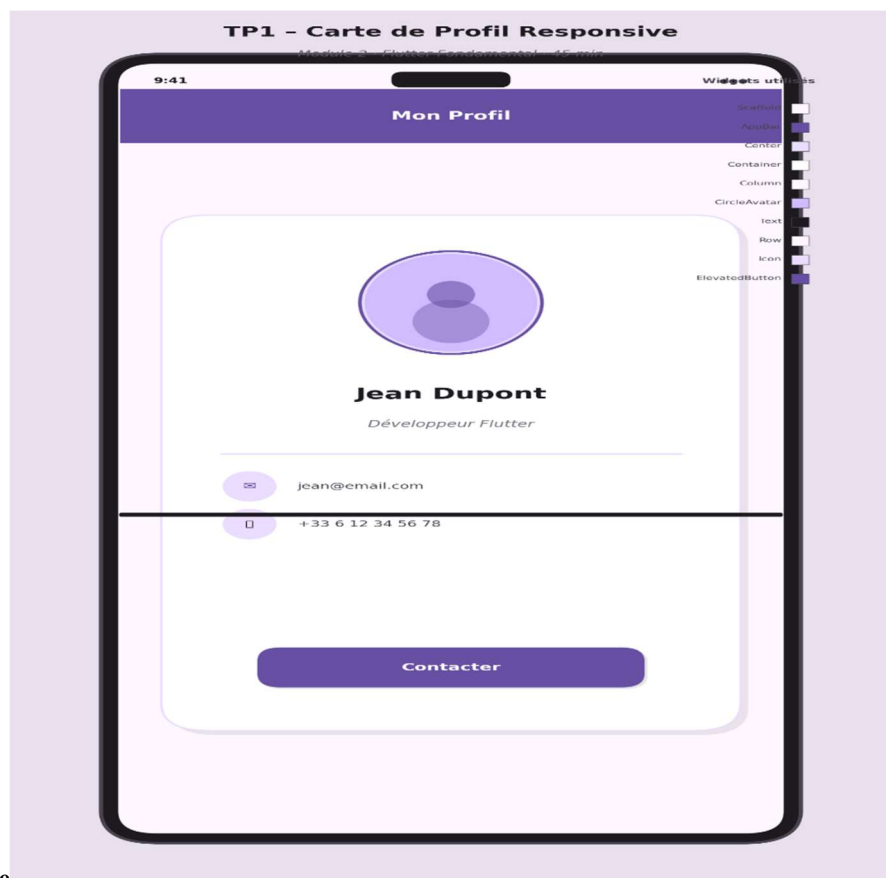
ÉNONCÉS DES TRAVAUX PRATIQUES

TP1 – Carte de Profil Responsive (45 min)

Contexte Vous créez une carte de profil utilisateur qui s'adapte à la taille de l'écran. C'est votre premier widget Flutter complexe.

Objectifs pédagogiques

- Combiner plusieurs widgets de base
- Utiliser Container, Column, Row, CircleAvatar, Text
- Comprendre le nesting (imbrication) de widgets
- Découvrir EdgeInsets et BoxDecoration



Maquette visuelle attendue

Consignes Étape 1 – Créer le projet

```
flutter create tp_carte_profil  
cd tp_carte_profil
```

Étape 2 – Modifier lib/main.dart Créez un StatelessWidget nommé CarteProfil qui affiche :

- Un Scaffold avec AppBar (titre : “Mon Profil”)
- Un Center contenant un Container de largeur fixe (300) ou responsive
- À l’intérieur du Container :
 - Un Column avec mainAxisAlignment: MainAxisAlignment.center
 - Un CircleAvatar avec radius: 50 et une image (utilisez NetworkImage avec une URL placeholder comme <https://i.pravatar.cc/150?img=11>)
 - Un SizedBox(height: 16)
 - Un Text avec le nom (style: fontSize: 24, fontWeight: FontWeight.bold)
 - Un Text avec le métier (style: fontSize: 16, color: Colors.grey)
 - Un SizedBox(height: 24)
 - Deux Row pour l’email et le téléphone (chacun avec un Icon et un Text)
 - Un SizedBox(height: 24)
 - Un ElevatedButton avec le texte “Contacter”

Étape 3 – Styler le Container

- Couleur de fond blanche
- Bord arrondi (BorderRadius.circular(16))
- Ombre portée (BoxShadow)
- Padding intérieur de 24

Étape 4 – Tester le responsive

- Lancez sur un émulateur téléphone
- Testez en mode paysage (rotation)
- Observez ce qui se passe. Pourquoi la carte ne s’adapte-t-elle pas parfaitement ?

Étape 5 – Amélioration (bonus)

- Remplacez la largeur fixe par MediaQuery.of(context).size.width * 0.85
- Ajoutez un SingleChildScrollView pour éviter les débordements en petit écran **Livrable attendu :**

Capture d’écran du rendu + code source main.dart.

TP2 – Démarrage du Fil Rouge : Écran d’Accueil (45 min)

Contexte Vous démarrez l’application fil rouge qui accompagnera toute la formation. Pour ce module, l’écran est **statique** (données en dur), mais structuré comme si elles venaient d’une vraie source.



Objectifs pédagogiques

- Structurer un écran complexe avec Scaffold
- Utiliser ListView pour une liste scrollable
- Créer un widget réutilisable pour les cartes
- Comprendre l'importance de const dans les widgets

Données factices (à intégrer en dur)

```
final List<Map<String, dynamic>> tachesFactices = [  
    {'id': '1', 'titre': 'Conception UI', 'categorie': 'Design', 'priorite': 'Haute', 'avancement': 0.75},  
    {'id': '2', 'titre': 'Intégration API', 'categorie': 'Backend', 'priorite': 'Moyenne', 'avancement':  
    0.30},  
    {'id': '3', 'titre': 'Tests unitaires', 'categorie': 'Qualité', 'priorite': 'Basse', 'avancement':  
    0.0},  
    {'id': '4', 'titre': 'Déploiement', 'categorie': 'DevOps', 'priorite': 'Haute', 'avancement': 0.10},  
    {'id': '5', 'titre': 'Documentation', 'categorie': 'Projet', 'priorite': 'Moyenne', 'avancement':  
    0.50},  
];
```

Consignes Étape 1 – Créer le projet fil rouge

```
flutter create fil_rouge_gestion  
cd fil_rouge_gestion
```

Étape 2 – Créer le widget CarteTache Créez un StatelessWidget réutilisable dans lib/widgets/ carte_tache.dart :

```
class CarteTache extends StatelessWidget {  
    final String titre;  
    final String categorie;  
    final String priorite; // 'Haute', 'Moyenne', 'Basse'  
    final double avancement; // 0.0 à 1.0  
  
    const CarteTache({  
        super.key,  
        required this.titre,  
        required this.categorie,  
        required this.priorite,  
        required this.avancement,  
    });  
  
    // ...  
}
```

Dans le build, retournez un Card contenant un Padding contenant un Column avec :

Une Row avec :

Un Container circulaire de couleur (rouge pour Haute, orange pour Moyenne, vert pour Basse)

- Un SizedBox(width: 12)

- Un Text avec le titre (style bold, 16)
- Un SizedBox(height: 8)
- Un Text avec la catégorie (style grey, 14)
- Un SizedBox(height: 12)
- Un LinearProgressIndicator avec la valeur avancement
- Un SizedBox(height: 4)
- Un Text affichant le pourcentage ($\{(\text{avancement} * 100).toInt()\}\%$)

Étape 3 – Créer l’écran d’accueil Dans lib/main.dart, créez un StatelessWidget PageAccueil qui retourne un Scaffold avec :

- AppBar : titre “Mon Tableau de Bord”, elevation: 0, couleur de fond blanche, texte noir
 - body : un Padding de 16 contenant une Column avec :
 - Un Text “Bonjour, Utilisateur ” (24, bold)
 - Un Text “Vous avez 5 tâches en cours” (16, grey)
 - Un SizedBox(height: 24)
 - Un Expanded contenant un ListView.builder qui affiche les 5 tâches factices
 - floatingActionButton : un FloatingActionButton avec l’icône Icons.add
- Étape 4 – Organiser le code** Créez la structure suivante dans lib/ :

```
lib/
├── main.dart
├── widgets/
│   └── carte_tache.dart
└── models/
    └── tache.dart (créez une classe Tache simple ici)
```

Étape 5 – Créer le modèle Tache Dans lib/models/tache.dart, créez une classe Tache avec les propriétés correspondant aux données factices. Utilisez final et un constructeur avec paramètres nommés.

Livrable attendu : Code source structuré + capture d’écran de l’écran d’accueil.

FICHE RÉCAP STAGIAIRE – Flutter Fondamental

Points clés à retenir

Table 6 – continued

Concept	Explication
Widget	Description immuable d’une partie de l’UI. Tout est widget.
Concept	Explication



StatelessWidget	Pas d'état interne. Ne change jamais après construction.
StatefulWidget	A un état mutable. Peut se reconstruire avec setState().
Scaffold	Structure de base d'une page (AppBar, body, FAB, drawer).
Container	Widget polyvalent : taille, couleur, padding, marge, décoration.
Row / Column	Organisation horizontale / verticale des enfants.
Hot Reload	Met à jour l'UI en gardant l'état. Indispensable en développement.
pubspec.yaml	Fichier de configuration : dépendances, assets, polices.
const	Optimise les performances. Utilisez-le sur les widgets qui ne changent pas.

Structure type d'une page Flutter

```
class MaPage extends StatelessWidget {  
  
  const MaPage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Titre')),  
      body: Padding(  
        padding: const EdgeInsets.all(16.0),  
        child: Column(  
          children: [  
            // Vos widgets ici  
          ],  
        ),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: () {},  
        child: const Icon(Icons.add),  
      ),  
    );  
  }  
}
```

Erreurs fréquentes à éviter

1. **Oublier const** sur les widgets statiques → perte de performance
2. **Nesting excessif** → code illisible. Extraire en widgets séparés.
3. **Largeur/hauteur fixes** → problèmes sur petits écrans. Privilégiez Expanded, Flexible, MediaQuery.

4. **Oublier pubspec.yaml** pour les images/polices → image non trouvée
5. **Confondre margin et padding** : margin = extérieur, padding = intérieur
6. **Ne pas utiliser super.key** dans le constructeur → warnings et perte d'optimisation

Check mentale avant de coder un écran

- ☐ Ai-je besoin d'un état qui change ? → StatefulWidget : StatelessWidget
- ☐ Ma page a-t-elle une AppBar ? → Scaffold
- ☐ Mes données sont-elles en liste ? → ListView.builder
- ☐ Ai-je des images/assets ? → pubspec.yaml à jour
- ☐ Mes widgets statiques ont-ils const ? → Performance
- ☐ L'écran est-il scrollable si contenu long ? → SingleChildScrollView / ListView

QUIZ DE VALIDATION – Module 2

Durée : 10 minutes – 8 questions

Question 1 (QCM)

Quelle commande permet de vérifier que Flutter est correctement installé ?

- A. flutter check
- B. flutter verify
- C. flutter doctor
- D. flutter status

Question 2 (QCM)

Quel widget est le point d'entrée racine d'une application Flutter Material ?

- A. MaterialApp
- B. Scaffold
- C. Container
- D. AppBar

Question 3 (Ouvrte)

Quelle est la différence entre StatelessWidget et StatefulWidget ? Dans quel cas utilisez-vous chacun ? **Réponse attendue :**

- StatelessWidget : widget sans état, immuable. Utilisé pour du contenu statique (texte, icône, image).
- StatefulWidget : widget avec état mutable. Utilisé quand l'UI doit changer (formulaire, compteur, liste dynamique).

Question 4 (QCM)

Quelle méthode appelle-t-on pour demander la reconstruction d'un StatefulWidget ?



- A. rebuild()
- B. setState()
- C. refresh()
- D. update()

Question 5 (Ouvrte)

Écrivez le code minimal d'un StatelessWidget nommé BoutonPrincipal qui affiche un ElevatedButton avec le texte "Cliquez ici".

Réponse attendue :

```
class BoutonPrincipal extends StatelessWidget {  
  const BoutonPrincipal({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: () {},  
      child: const Text('Cliquez ici'),  
    );  
  }  
}
```

Question 6 (QCM)

Dans pubspec.yaml, où déclare-t-on les images locales utilisables dans l'application ?

- A. Dans dependencies
- B. Dans dev_dependencies
- C. Dans la section flutter > assets
- D. Dans environment

Question 7 (QCM)

Quel widget permet d'organiser des enfants horizontalement ?

- A. Row
- B. Column
- C. Stack
- D. Wrap

Question 8 (QCM – difficulté +)

Que se passe-t-il si vous modifiez une variable d'état dans un StatefulWidget sans appeler setState() ?

- A. L'application plante

- B. La valeur ne change pas
- C. La valeur change en mémoire mais l'UI n'est pas mise à jour
- D. Le widget est détruit

Barème indicatif

- 8/8 : Prêt pour le Module 3 (UI avancée)
- 6-7/8 : Acquis solides, réviser le cycle de vie des StatefulWidget
- 4-5/8 : Refaire les TP, le Module 3 sera difficile
- < 4/8 : Revoir la théorie widgets et refaire TP1

- **Documentation Flutter** : <https://docs.flutter.dev>
- **Widget catalog** : <https://docs.flutter.dev/ui/widgets>
- **DartPad Flutter** : <https://dartpad.dev> (mode Flutter)
- **Material Design 3** : <https://m3.material.io>
- **Icônes Flutter** : <https://fonts.google.com/icons>